Securing QR Codes Infrastructure Using AI to Detect Malicious Activity

Luis Rivas
US Army
Johns Hopkins University

Varun Kumar Singh
Information Security Institute
Johns Hopkins University

Vinayak Khandelwal
Information Security Institute
Johns Hopkins University

Lanier Watkins
Information Security Institute
Johns Hopkins University

2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC) | 979-8-3315-0769-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/CCWC62904.2025.10903910

***Abstract*—With the advent of QR Codes, mobile devices, and computers, users can exchange information quickly but not securely. QR Codes-based attacks are rising rapidly, threatening the trust of this extensively used technology. Whether you use it at a restaurant, conference, or payment station, QR Codes inherently rely on user trust to safeguard transactions. Phishing scams using fake codes have increased by over 50% , while using QR vector malware attacks have increased by over 2400% since 2020 [3]–[5]. These alarming results suggest that integrity protections must be reinstated to restore public trust. This research addresses these challenges by leveraging Artificial intelligence (AI) and third-party services to detect QR Codes-based threats. We propose a Secure QR Codes system employing anomaly detection, risk signaling, and four machine-learning models trained on standard and malicious QR Codes. Using encryption engines, AI-based predictions, Application Programming Interface (API), proxy services, and AI assessment modules, the integrated solution identified over 96% simulated real-world attacks like phishing, exfiltration, injection, and code execution. Our approach detects QR Codes with modified structures. It provides a practical, non-proprietary, web-deployable solution to predict and mitigate these threats, restoring trust in QR Codes systems across various applications.**

***Index Terms*—QR Codes, QR Code Security, Cybersecurity, Anomaly Detection, Secure QR Code Infrastructure, Machine Learning (ML), Artificial Intelligence (AI)**

I. Introduction

Quick Response (QR) Codes have become ubiquitous for rapidly accessing digital information via smartphone scanning. However, vulnerabilities such as phishing links, malware injection, and data theft pose significant threats to privacy and security, necessitating robust defenses. While cryptographic and software protections exist, they often fail to adapt to increasingly sophisticated attacks.

In this initial study, we investigate the integration of simple Artificial Intelligence (AI) and Machine Learning (ML) based models to autonomously identify current QR Codes threats. By training ML models on benign and malicious QR Codes datasets, we aim to distinguish legitimate ones from those with manipulated structures, redirects, or harmful payloads. The research aims to develop a multilayered secure QR Code infrastructure that combines data analysis, software protections, and AI-based models to enhance the robustness of solutions against evolving attacks.

979-8-3315-0769-5/25/$31.00 ©2025 IEEE

QR Codes are two-dimensional barcodes that optical scanners and camera devices can decode. They offer advantages over linear barcodes through error correction capabilities, allowing data recovery from partially damaged images [1]. Despite their widespread use and ease of access, QR Codes lack inherent security features. The plaintext storage model leaves sensitive personal or financial data vulnerable to unauthorized access and manipulation [2]. The absence of authentication or provenance verification enables adversaries to replace legitimate codes with malicious ones, directing users to hazardous sites or delivering malicious payloads.

To address this gap, we present the AP3X Secure QR Codes Infrastructure—an open-source, operating system-agnostic solution that does not require installing a new application. Users can scan or upload QR Codes and receive predictions about its safety. Our approach relies on extracting 24 lexical features and employing code-based logic to determine the malicious nature of the encoded message. The integrated infrastructure achieved a detection rate of 100% accuracy for four crafted QR Codes-based attacks and a 96% success rate using AI-based prediction [3]–[5].

This paper is structured as follows: Section 2 provides background information on QR Codes, and Section 3 reviews related work in the field. Section 4 describes four crafted QR Codes that exploit confidentiality, integrity, or availability. Section 5 details the AP3X Secure QR Codes Infrastructure, including the engineering of the web application and AI ML modules. Section 6 examines the experimental process for designing and implementing the secure solution, Section 7 presents the results and discussion, and Section 8 outlines the limitations of our research. Finally, Section 9 concludes the paper and suggests future work.

II. Background and QR Codes Vulnerabilities

QR Codes have become ubiquitous for encoding and quickly sharing information via smartphone scanning. They are two-dimensional barcodes decodable by optical scanners and camera devices, offering advantages such as error correction capabilities that allow data recovery from partially damaged images [1]. Despite their widespread use in areas like healthcare, payments, and transit, QR Codes lack inherent security features, making them vulnerable to cyber-attacks [7]–[12].

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.

Several key security issues and vulnerabilities have been identified:

Plaintext Storage: QR Codes store data in cleartext without encryption, potentially exposing confidential information. This allows unauthorized access to sensitive personal details, healthcare records, and financial information [9]. Adversaries can exploit leaked data for phishing, fraud, and identity theft.

Lack of Data Integrity Protections: Without digital signatures or authentication mechanisms, attackers can modify legitimate codes to redirect users to phishing sites or embed malware within payload data, compromising device security. This inability to authenticate the QR Code threatens data integrity.

Partial Code Manipulation: Minor modifications to QR Codes modules can alter encoding modes, payload lengths, or control characters, leading to misinterpretation. Attackers can inject malicious payloads into an intact encoded message.

Variability in Reader Software: Inconsistent capabilities among scanning and decoding software introduce weaknesses. Reliance on reader programs with inadequate threat detection allows attackers to exploit vulnerabilities. Some applications compromise users' privacy by transmitting scanned data to external servers, eroding public confidence and hindering technology adoption.

Lack of Sender Authentication: Absence of mechanisms to verify the origin of a code allows for spoofing attacks, where attackers replace legitimate codes with malicious ones.

Limited Error Correction Against Malicious Errors: While QR Codes have some redundancy for restoring damaged codes, they are not resilient against malicious errors inserted by attackers.

As a result, QR Codes serve as vectors for diverse cyberattacks, including:

1) **Phishing Campaigns:** Fake QR Codes misdirect users to adversary-controlled sites, leading to credential theft.
2) **Malware Propagation:** Infected payloads deliver spyware or Trojans to scanning devices.
3) **Data Interception:** Man-in-the-middle attacks intercept QR data on insecure channels.
4) **Data Exfiltration:** Accessing and decoding unencrypted QR payloads to steal personal and financial information.
5) **Monetary Theft:** Manipulating QR Codes for ticketing, coupons, and payments fraud.

These vulnerabilities undermine user trust and adoption, highlighting the imperative to develop robust QR Codes security mechanisms that ensure confidentiality, integrity, and availability. This research addresses these vulnerabilities by proposing effective defenses to restore confidence in QR Codes systems across various industries.

III. Related Works

QR Codes have been increasingly utilized in various applications, from electronic ticketing systems to secure information sharing. Finzgar et al. [13] developed a QR Codes-based ticketing system to prevent fraud. Vidas et al. [14] found that most people scan random QR Codes out of curiosity, which attackers can exploit. Kharraz et al. [15] highlighted real-world risks of QR Codes attacks spreading phishing links and malware.

Several solutions have been proposed to enhance QR Codes security. Mavroeidis and Nicho [16] designed a cryptographic system to ensure QR Codes authenticity and integrity, although it lacks widespread implementation due to QR Codes variety. Ahamed and Mustafa [12] proposed SQRC, providing confidentiality via encryption and integrity via signatures. Focardi et al. [10] studied QR Codes usability, finding that while signature schemes can be problematic, certificates help balance usability and security. Yadav et al. [17] developed a QR reservation system, demonstrating usability incentives driving adoption.

Our research focuses on securing encoded messages in QR Codes using a unique infrastructure that extracts features and performs static analysis, aiming to address the limitations of previous solutions.

QRCS (Quick Response Code Secure) Tool: Mavroeidis and Nicho [16] introduced the QRCS platform, which employs server-side restrictions and ECDSA to produce digitally signed QR Codes. However, it requires a specialized reader to decode and verify signatures, limiting its widespread usability compared to the AP3X Secure QR Solution.

QR Decoder: Kharraz et al. [15] and Vidas et al. [14] identified security risks in QR Codes due to ease of malicious modifications and inadequate threat detection by reader software. Attackers can alter QR Codes by modifying pixels or encoding schemes, leading to data misinterpretation or malware distribution. To address these vulnerabilities, we developed a web application that analyzes QR Codes features to predict malicious intent.

SQRC (Secure QR Codes): Ahamed and Mustafa [12] proposed SQRC to ensure authentication and integrity in QR Codes using RSA cryptography and digital signatures through two models:**(1)Validation model**: Securely transfers confidential data using RSA encryption. **(2)Verification model**: Verifies information using RSA digital signatures. In contrast, our AP3X Secure QR Infrastructure analyzes encoded messages for acceptable risk levels rather than ensuring authentication between the user and the QR Codes.

QsecR: Secure QR Codes Scanner: In [33], the authors propose QsecR, a secure QR Codes scanner for Android devices that detects malicious URLs using 39 features across four categories. It achieved 93.5% accuracy on 4,000 real-world URLs. Our solution expands on this by utilizing lexical features and machine learning to identify URLs, payloads, and redirections, providing QR Codes analysis accessible from any device with web access. We trained multiple ML models on over 600,000 URLs to enhance user decision-making and expand the availability of the AP3X Secure QR solution.

IV. QR Codes: Crafting Attack Vectors

To validate the efficacy of our proposed solution, this research implemented previously devised QR Codes-based cyber attacks representing prevalent threats in this domain. The

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.

attacks were designed to evaluate mitigation capabilities across various vectors, including data exfiltration, payload injection, drive-by downloads, and cross-site scripting.

(1) QR Codes Exfiltration Attack QR Codes can be exploited to exfiltrate user data and device information when scanned. Encoding a URL redirect in the QR Codes to an exfiltration server under attackers' control can collect data points like geolocation, device details, and network information without the user's knowledge or consent. The exfiltration server aggregates this data to develop scanned user profiles for potential surveillance or future spear phishing attempts. While QR Codes provide convenience, transparent policies and consent mechanisms are needed to leverage them for ethical data collection.

(2) Malicious Payload Injection Given QR Codes' generous 2953-byte capacity, attackers can completely replace the encoded content to inject executables or scripts for malware distribution [18], [19]. Using a QR encoding library, the attacker first hides the malicious code like a weaponized PDF or Windows executable within an innocuous-looking QR Code [6]. When an unwitting user scans this code, the malware payload is extracted and triggered, compromising the device. This technique bypasses traditional perimeter defenses, enabling insiders to covertly spread malware using printed QR Codes by leveraging the ubiquitous smartphone scanning habit. Strong input validation and anomaly detection are needed to counter payload attacks.

(3) QR Codes-based Drive-by-Download By modifying QR Codes pixel modules or generating fake codes, attackers can redirect users to phishing sites through encoded links [9]. This relies on a lack of user awareness and consent mechanisms when handling QR Codes redirects. The recent Brave browser vulnerability enabled seamless redirection to malicious sites upon scanning QR Codes, highlighting the need for security prompts to avoid drive-by attacks [20]. As human inspection cannot validate QR Codes intent, output sanitization alongside warnings can help prevent exploits through tampered redirects.

(4) UXSS via QR Codes Javascript Execution Unsafe handling of QR Codes input can trigger critical web vulnerabilities like XSS, as demonstrated by the *Firefox Android* UXSS bug [20] [21] [22]. By encoding *JavaScript* code prefixed with "*javascript:*" rather than "*http://,*", attackers executed scripts when the QR Codes was scanned. While blocking dangerous prefixes mitigates this issue, treating QR Codes data with server-side input security in mind across the stack can prevent injection risks, either through manual review or automated detection. These examples demonstrate the breadth of potential QR Codes exploits from exfiltration, malware distribution, phishing, and code injection. A combination of safe engineering practices, anomaly detection, user awareness, and organizational governance is imperative to address the diverse security risks posed by QR Codes.

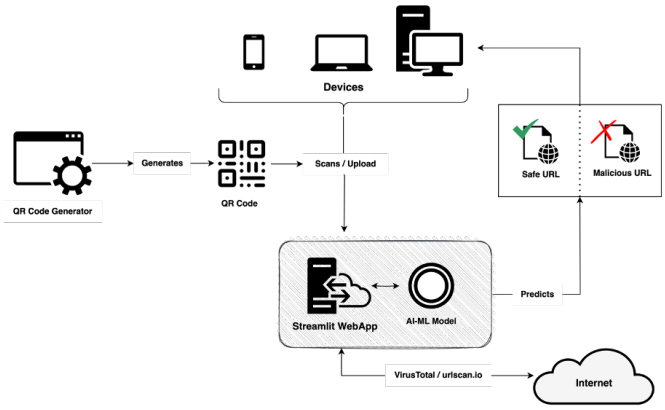

Fig. 1. Secure QR Codes Infrastructure Design

V. AP3X Secure QR Codes Infrastructure

A. AI-Based Detection of QR Codes Attacks

Despite its widespread use in information sharing, authentication, and payments, QR Codes lack integrity protections, making them vulnerable to attacks such as phishing, SQL injection, and malware distribution [7], [14]. Common attack vectors include:

1) **Data exfiltration**.
2) **Complete replacement with malicious code**.
3) **Injecting a malicious payload into an encoded message**.

We created a dataset of benign and malicious QR Codes to evaluate AI-based detection of these attacks, extracting visual features like pixel distribution and shape for training machine learning models [23], [24]. These models were tested on new attack types to assess generalization, with metrics such as accuracy, precision, recall, and F1-score providing insights into model robustness. Our experimental framework—dataset generation, attack simulation, and model evaluation—helped identify practical multi-vector QR Code malware detection approaches. We observed a gap in current QR Code security, as most scanners reveal only basic URL information without evaluating content risk. To address this, we developed a secure infrastructure (Figure 1), allowing users to determine a QR Code's safety based on various factors. Our system includes a web application (Figure 2) that detects malicious QR Codes from user-uploaded images, or webcam captures, processing the data with ML models and integrating API analyses from VirusTotal and urlscan.io for a comprehensive security assessment.

B. User Interface

The user interface features simple navigation tabs that allow users to interact with the application's functionalities. Based on their selection, users are provided with interfaces to submit QR Codes: **(1) QR Codes Image Prediction:** Users can upload images in PNG, JPG, or JPEG format by dragging and dropping files or using the Browse file' button (Figure 2). After selecting a file, they click the Predict' button to process

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.

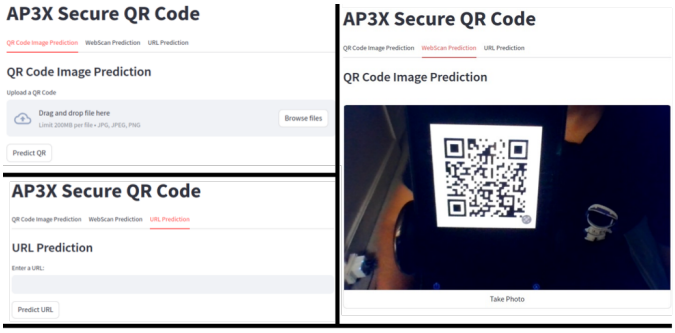

Fig. 2. AP3X Secure QR Codes: Prediction using a QR Codes and URL

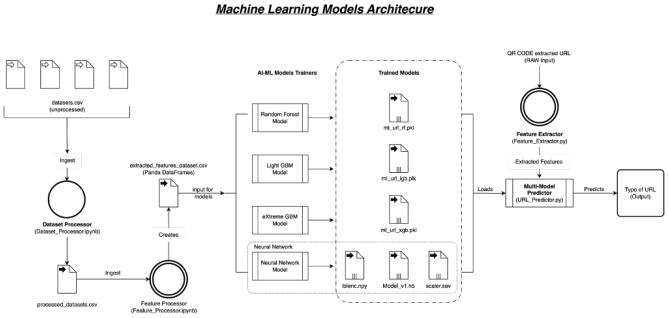

Fig. 3. Machine Learning Models Architecture

it. **(2) WebScan Prediction:** Users access a real-time video feed from their device's camera and can capture an image by clicking a button. The captured image is then processed using the 'Predict' button. **(3) URL Prediction:** Users can enter a URL into a text box and click the 'Predict' button to process it (Figure 2).

Processed data is displayed in two sections: *QR Codes data & Reports* and *Trust metrics*. The first section shows detailed information extracted from the QR Codes and various APIs. The second section provides key highlights from APIs and ML models in a side drawer, helping users make informed decisions and alerting them to any detected malicious content.

C. Processing

The submitted file or image is processed using *Python* libraries to extract information and enumerate URLs. If no URLs are found, a warning message "*non-URL content detected*" is displayed, and the file is sent to VirusTotal for analysis. Otherwise, extracted URLs are input into ML models (XGBoost, LightGBM [29], Random Forest, Neural Network) for prediction and scanned using VirusTotal and urlscan.io. The final verdict helps users make informed decisions.

D. Use of Third-Party Services

VirusTotal: An aggregator of antivirus products and online scan engines, allowing users to scan suspected URLs or files for viruses and verify false positives [25]. We use this service to analyze data extracted from users' QR Codes.

urlscan.io: A free service to scan and analyze websites. When a URL is submitted, it automates browsing and records activities, collecting data such as domains, IPs, screenshots, DOM content, *JavaScript* variables, and cookies [26], [27].

E. Data Handling

The website uses the *Streamlit Python* library for front-end and back-end management. According to the documentation, "The data is copied to the *Streamlit* backend via the browser and contained in a *BytesIO* buffer in *Python* memory (i.e., RAM, not disk)" [28]. Since the data resides only in memory, it is automatically deleted when no longer needed. In our application, files are deleted from memory when a user session ends—either by closing the browser tab or reloading the webpage—reducing data handling overhead and mitigating potential data-retention-based attacks.

TABLE I
EXTRACTED FEATURES FOR ML MODELS

Feature Description
having_ip_address: Checks if the URL contains an IP address.
abnormal_url: Determines if the URL lacks a hostname.
google_index: Checks if the URL is indexed by Google.
count_dot: Occurrences of '.' in the URL.
count_www: Occurrences of 'www' in the URL.
count_atrate: Number of '@' characters in the URL.
count_dir: Number of directory paths in the URL.
count_embed_domain: Occurrences of '//' in the path .
shortening_service: Use of known URL Shortening Service.
count_https: Number of times 'https' appears in the URL.
count_http: Number of times 'http' appears in the URL.
count_percent: Number of '%' characters in the URL.
count_question_mark: Number of '?' characters in the URL.
count_hyphen: Number of '-' characters in the URL.
count_equal: Number of '=' characters in the URL.
url_length: Measures the total length of the URL.
hostname_length: Measures the length of the hostname in the URL.
suspicious_words: Presence of suspicious words in the URL.
digit_count: Number of digits in the URL.
letter_count: Number of alphabetic characters in the URL.
fd_length: Measures the length of the first directory in the URL Path.
tld_length: Measures the length of the top-level domain in the URL.
entropy: Calculates the Shannon entropy of the URL.
has_port: Checks if the URL has a port number.

Table I outlines the key features engineered from the URLs in the dataset used to train all the AI models for detecting malicious QR Codes. In total, 24 distinct features that quantify different attributes of the URLs were extracted.

F. Artificial Intelligence Machine Learning Modules

1) Model Architecture and Dataset Preparation: The web application operates within a Docker container, integrating components via a *Streamlit*-based frontend and backend APIs. *Python* libraries like *Pandas* and *Itertools* facilitate efficient processing of over 600,000 URLs from multiple datasets (Figure 3). The datasets used for training and testing the machine learning models were compiled from: *(1) ISCX-URL-2016 Dataset*: Primary dataset with various URL types—benign, phishing, malware, and defacement [34]. *(2) Malware Blacklist Domains Dataset*: Additional phishing and malware URLs from Malware Domain Blacklist [35]. *(3) Malicious URLs Dataset*: Benign URLs from the Faizan Git repository for

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.

balancing [36], [37]. *(4) Phishing Site URLs Dataset*: More phishing URLs from PhishTank and PhishStorm [38]. *(5) Malicious and Benign URLs Dataset*: Supplementary benign and malware URLs [39]. Datasets were standardized with consistent category labels ('benign', 'defacement', 'phishing', 'malicious') and merged into a single DataFrame using *pd.concat()*, creating a diverse dataset for accurate malicious URL classification.

2) Feature Extraction: We extracted features such as URL structure, lexicon, entropy, and keywords (Table I) to enable precise classification of unseen QR Codes based on embedded URLs. These features are critical for detecting malicious URLs used in QR Codes because they identify suspicious patterns often used by threat actors to evade detection and deceive users. Based on threat intelligence metrics, malicious URLs are mostly never consistent and need frequent modifications, to evade detection. This deviates them from standard structures by incorporating unusual elements such as IP addresses, excessive special characters, long or obfuscated paths, suspicious keywords, or uncommon port numbers. Features like URL length, entropy, and directory depth reveal obfuscation or complexity aimed at hiding malicious content, while checks for Google indexing and shortening services highlight URLs that lack credibility or transparency. Together, these features provide a robust framework for flagging these potential threats.

3) Machine Learning Models: Four machine learning models were trained: Random Forest (RF), Light Gradient Boosting Machine (LightGBM), XGBoost, and Neural Network (NN). The data was split 80/20 into training and test sets for robust performance evaluation [30].

Random Forest (RF): An ensemble method combining decision trees, resilient to overfitting and providing high accuracy. The RF model achieved 96.7% accuracy (Table IV), with high true positive rates across all classes, especially in detecting defacement and phishing URLs. Improvement is needed in distinguishing benign from malicious URLs.

LightGBM: A gradient boosting framework optimized for efficiency and scalability. The LightGBM model attained 94.4% accuracy, showing strong detection of defacement and phishing URLs but requiring better separation between benign and malicious URLs.

XGBoost: An optimized gradient boosting library focusing on speed and performance. The XGBoost model achieved 94.7% accuracy, with high true positive rates, particularly for defacement and phishing URLs. Distinguishing benign from malicious URLs can be improved.

Neural Network (NN): Inspired by biological neural networks for pattern recognition [31]. The NN model reached 95.0% accuracy, demonstrating strong detection of defacement and phishing URLs. Further refinements could enhance the classification between benign and malicious URLs.

VI. Experimental Evaluation

The research also focused on developing two experiments using the AP3X Secure QR Codes Infrastructure. Section A discusses the infrastructure for developing and implementing the web application with the ML models. Sections B and C describe the experimental evaluation using QR-crafted Attacks and randomly selected QR Codes, respectively.

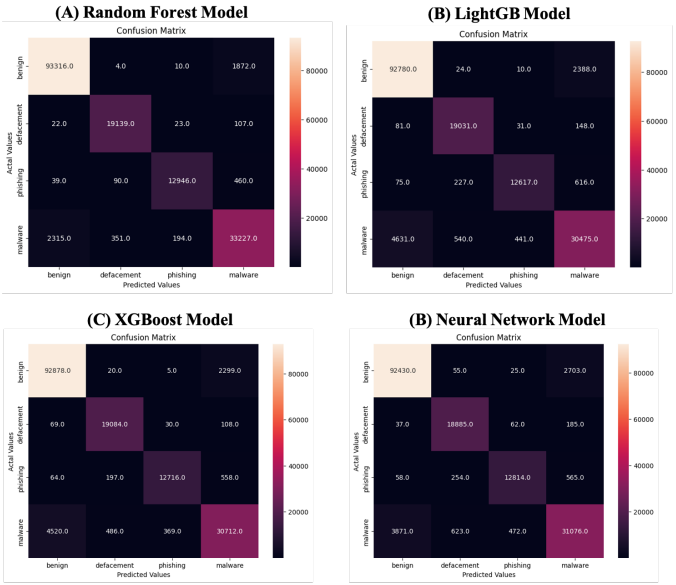

Fig. 4. Confusion Matrices for ML Models. Confusion matrices illustrating the classification performance of Random Forest, LightGBM, XGBoost, and Neural Network models on four URL categories: benign, defacement, phishing, and malware. The matrices display true positive rates along the diagonal, with ensemble models demonstrating higher robustness and reduced misclassification rates compared to the neural network, particularly in distinguishing phishing and malware.

TABLE II
Performance Metrics of Different Models

Model	Accuracy	Precision	Recall	F1 Score
RF	0.967	0.97	0.96	0.96
LGB	0.944	0.95	0.93	0.96
XGB	0.947	0.95	0.94	0.94
NN	0.950	0.94	0.94	0.94

Table II compares the overall performance metrics of four different ML models, including accuracy, precision, recall, and F1 score.

A. Experimental Setup

The web application and models were hosted inside a docker container running on Ubuntu 22.04 Operating system with a 12th Gen Intel Core i7 processor where we provision 8GB RAM and 4 cores for the application. We used *Python 3.10+* as the interpreter for the project with which we used third-party libraries like *Streamlit*, *pyzbar*, *pillow*, *requests*, and *urlextract* to name a few. In contrast, the ML models use *scikit-learn, lightgbm, xgboost, tensorflow, googlesearch-python* and more. We used Oneplus 7T HD1901 with Android 12 Oxygen OS 12.1 and iPhone 14 Pro MQ0N3LL/A with iOS 17.1 for simulation and carrying out the QR Codes-based attacks.

B. Simulating the QR Codes Attacks

Specific attacks include exfiltration servers that gather user info before redirecting, hiding malware inside QR Codes

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.

that compromise devices when scanned, exploiting browser weaknesses to force malicious site visits and downloads, and executing *JavaScript* like XSS payloads when users authorize QR Codes execution.

QR Codes Exfiltration Attack: We generated a QR Codes with a PHP-Laravel project [41] using any arbitrary endpoint since data exfiltration occurs because of a custom implementation of a QR Codes generator, which involves encoding exfiltration url in the QR Codes and then redirecting the user to required url afterward.

Malicious Payload Injection: A command line tool *qrencode* allowed us to inject and generate QR Codes with a Windows executable. This executable was generated using the Metasploit framework *msfvenom* with a simple payload that opened a new window and displayed a text message. We can recover the executable with *zbarimg*, another command line tool *zbar-tools*.

QR Codes-based drive-by-download: For this attack to work, we downloaded the vulnerable version of Brave browser v1.50.114 based on Chromium 112.0.5615.49 with the vulnerability and generated a QR Codes with a direct download URL pointing to a file hosted in Google Drive [32]. QR Codes Generator [40] was used to generate the QR Codes for this attack.

UXSS via QR Codes JavaScript Execution: In the identified Universal Cross-site Scripting (UXSS) vulnerability affecting Mozilla Firefox for Android version 93.2.0, for the exploit, we generated the QR Codes with a URL address prefix of '*javascript:*' instead of the conventional '*http*:' or '*https:*' with a QR Codes generator [40].

Once we had the malicious QR Codes, we scanned and uploaded them to our solution. Then, the AP3X Secure QR Codes Infrastructure assesses these QR Codes by extracting data, processing it, and then, based on analysis, providing a verdict. This verdict is based on analysis and final prediction from different ML models and APIs, namely VirusTotal and urlscan.io.

C. Dataset of Randomly selected QR Codes and Green Threads-based Hyper-looped tool for evaluation

We randomly selected over a thousand QR Codes to evaluate the infrastructure. We designed an asynchronous framework that employed green threads in our research to assess the efficacy of our suggested solution on the above QR Codes dataset. By employing *Python's greenlets*, we improved the efficiency and speed of the interface, allowing for non-blocking testing activities. Within this configuration, the tool iteratively traversed the dataset, extracting and scrutinizing the prediction outcomes for each QR Codes, enabling us to better evaluate the solution's precision and resilience in detecting potential security risks associated with the QR Codes.

VII. Results and Discussions

The comprehensive defense strategy involves extracting QR Codes features, differentiating URL and non-URL information, and predicting with AI models, urlscan.io, and VirusTotal for URL data. We compress non-URL data and analyze it with VirusTotal. This approach effectively mitigates the risks associated with QR Codes-based attacks we mentioned in Section 4, with results discussed below.

TABLE III
Experimental results of QR Codes-based attacks using Mobile devices

QR Attack	Without Security	With AP3X Secure QR
Exfiltration Attack	Successful	Redirection
Payload Injection	Successful	Executable
Drive-by-download	Successful	Malicious
UXSS JS Execution	Successful	Unknown

Table III shows the experimental results using the OnePlus 7T and iPhone 14 Pro, demonstrating the effectiveness of the proposed AP3X Secure QR solution against four QR Codes-based attacks.

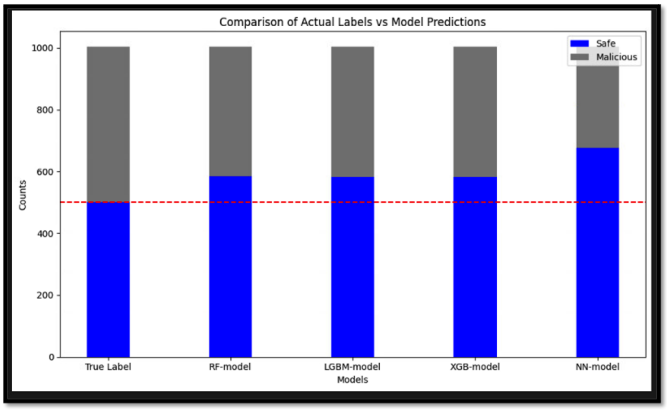

Fig. 5. Graphical Comparison of the Actual Labels vs ML Model Predictions for safe and malicious URLs

A. QR Codes-based Attacks vs AP3X Secure QR Codes Infrastructure

The experiment evaluated the response of the AP3X Secure QR Codes Infrastructure against QR Attacks for (i) Exfiltration, (ii) Payload injection, (iii) Drive-by-Download, and (iv) UXSS JS Execution. Each attack was identified by the AI modules or logic programmed in the infrastructure. Table III shows the result of each attack and the complete effectiveness.

B. Random QR Codes vs AP3X Secure QR Codes Infrastructure

The experimental results evaluated the AP3X Secure QR Codes Infrastructure using a selected dataset of 497 QR Codes labeled as 'safe' and 503 labeled as 'malicious' by trusted sources, including VirusTotal and urlscan.io. This dataset was constructed to accurately replicate the conditions and dangers faced in real-world settings, thereby creating a strong foundation for testing. We utilized an advanced hyper-looped tool in combination with the AP3X Secure QR solution to carry out and evaluate each QR Codes (Figure 5). Our methodology was designed to identify the inherent risks and

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EXPERIMENT METRICS FOR ML MODEL PREDICTIONS OF RANDOM URLS

Metrics	RF-Model	LGBM-Model	XGB-Model	NN-Model
Accuracy	0.91625	0.91326	0.91724	0.72183
Precision	1.00	0.99	1.00	0.84
Recall	0.83300	0.83101	0.83499	0.54870
F1-Scores	0.90889	0.90574	0.91007	0.66425

Table IV shows the overall prediction correctness (Accuracy) is low compared to others, with a moderate rate of correctly predicted positives (Precision), a poor rate of correctly identifying all positives (Recall), a subpar harmonic mean of Precision and Recall (F1-Score), and a high number of both types of prediction errors.

evaluate the resilience and accuracy of our detection systems under simulated operational conditions.

Table IV shows the performance metrics in which the Random Forest (RF) model excels in recall (0.83300) and F1-score (0.90889), making it suitable for scenarios requiring high true positive detection, though it risks misclassifying benign URLs. LightGBM balances precision (0.99) and scalability, favoring environments where minimizing false positives is crucial, though its recall (0.83101) suggests it may miss some threats. XGBoost delivers the highest accuracy (0.91724) and precision (1.00), ideal for applications needing strong overall reliability and low false positives but at the cost of a moderate recall (0.83499). In contrast, the Neural Network underperforms across most metrics, with low recall (0.54870) and accuracy (0.72183), highlighting the need for further optimization. Gradient-boosted models like XGBoost and LightGBM are preferable for precision-focused tasks, while RF is optimal for balanced detection in high-risk environments. A hybrid approach combining these strengths could further enhance robustness.

VIII. LIMITATIONS

While our ML models showed promising results, real-world deployment faces challenges. A primary concern is that malicious actors could attack AI systems by carefully altering inputs to bypass defenses; our study did not assess the models' resilience to such adversarial attacks or data poisoning methods. Future work should include comprehensive evaluations of the Secure QR Codes Infrastructure against these threats.

Additionally, reliance on third-party APIs like VirusTotal and urlscan.io introduces limitations due to query restrictions and associated costs, potentially affecting the models' ability to obtain real-time URL features. Our security enhancements focused more on integrity and privacy risks, with less emphasis on availability risks, such as denial-of-service attacks that could disrupt scanning systems. Further research should explore a wider range of threats, test the system in real-world settings, ensure compatibility with various QR Codes types, enhance integration, formally verify encryption security, address usability issues, and assess robustness against adversarial ML.

Moreover, given the rapidly evolving cybersecurity landscape, the datasets used for training may be outdated. Incorporating new datasets and features—such as live data extraction, content-based attacks, domain registration details, and similar URL characteristics—could improve model predictions.

Scaling the research for real-world deployment is challenging due to computational overhead, system latency, and reliance on external APIs, which introduce cost, query limitations, and potential single points of failure. The diverse QR Codes standards across industries further complicate adaptability, requiring modular designs and tailored detection methods. Future research should focus on decentralized approaches, such as edge-based detection and federated learning, to enhance scalability and reduce external dependencies while incorporating real-time threat intelligence to maintain system robustness.

IX. CONCLUSION AND FUTURE WORK

This research demonstrated that integrating AI and ML with traditional security controls enhances protection in QR Codes systems amid escalating cyber threats. The four models tested—Random Forest, LightGBM, XGBoost, and Neural Network—achieved average accuracies exceeding 93% in detecting simulated attacks, including QR Codes exfiltration, payload injection, phishing redirection, and *JavaScript* execution threats that compromised unprotected systems.

Future work could improve the Secure QR Codes Infrastructure by decoupling the ML models from the web server architecture; using an orchestrator like Kubernetes may enhance efficiency and allow model updates without affecting server operations. Exploring different AI models might yield better prediction accuracies for detecting QR Codes-based attacks.

In conclusion, our findings reveal that ML can adapt protections to counter emerging exploit tactics targeting QR Codes when combined with integrity checks, AI-based predictions, APIs, proxy filtering, and anomaly detection engines. As cyber threats continue to evolve, augmenting human governance with AI that scrutinizes distributed data vectors promises sustained resilience. Further research into adversarial ML is necessary to strengthen defenses as threat actors develop countermeasures.

REFERENCES

[1] H. Huang, F. Chang and W. Fang, "Reversible data hiding with histogram-based difference expansion for QR Codes applications," *IEEE Transactions on Consumer Electronics,* vol. 57, *(2),* pp. 779-787, 2011.

[2] N. P. A. Karniawati *et al*, "Community Perception of Using QR Codes Payment in Era New Normal," *PalArch's Journal of Archaeology of Egypt/Egyptology,* vol. 18, *(1),* pp. 3986-3999, 2021.

[3] (November 15). "Be careful what you scan: QR scams increase by 51%". Available: https://cybernews.com/news/qr-code-phishing-scams-quishing/.

[4] (October 27,). "The Rise in QR Codes Attacks." Available: https://www.digit.fyi/comment-the-rise-in-qr-code-attacks/.

[5] (November 21,). "The alarming rise of quishing is a red flag for CISOs." Available: https://www.csoonline.com/article/1248084/the-alarming-rise-of-quishing-is-a-red-flag-for-cisos.html

[6] (October 29,). "Surge in QR Codes Quishing: Check Point Records 587% Attack Spike." Available: https://www.hackread.com/qr-code-quishing-check-point-attack-spike/.

[7] K. Krombholz et al, "QR Codes security: A survey of attacks and challenges for usable security," in *Human Aspects of Information Security, Privacy, and Trust: Second International Conference, HAS 2014, Held as Part of HCI International 2014, Heraklion, Crete, Greece, June 22-27, 2014. Proceedings 2, 2014.*

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.

[8] A. Pawar et al, "Secure QR Codes scanner to detect malicious URL using machine learning," in *2022 2nd Asian Conference on Innovation in Technology (ASIANCON), 2022*.

[9] P. Kieseberg et al, "QR Codes security," in *Proceedings of the 8th International Conference on Advances in Mobile Computing and Multimedia, 2010*.

[10] R. Focardi, F. L. Luccio and H. A. Wahsheh, "Usable security for QR Codes," *Journal of Information Security and Applications*, vol. 48, pp. 102369, 2019.

[11] X. Han et al, "Medusa attack: Exploring security hazards of {in-app}{QR} code scanning," in *32nd USENIX Security Symposium (USENIX Security 23), 2023*.

[12] M. S. Ahamed and H. A. Mustafa, "A secure QR Codes system for sharing personal confidential information," in *2019 International Conference on Computer, Communication, Chemical, Materials and Electronic Engineering (IC4ME2), 2019*.

[13] L. Finžgar and M. Trebar, "Use of NFC and QR Codes identification in an electronic ticket system for public transport," in *SoftCOM 2011, 19th International Conference on Software, Telecommunications and Computer Networks, 2011*.

[14] T. Vidas et al, "QRishing: The susceptibility of smartphone users to QR Codes phishing attacks," in *Financial Cryptography and Data Security: FC 2013 Workshops, USEC and WAHC 2013, Okinawa, Japan, April 1, 2013, Revised Selected Papers 17, 2013*.

[15] A. Kharraz et al, "Optical delusions: A study of malicious QR Codes in the wild," in *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, 2014*.

[16] V. Mavroeidis and M. Nicho, "Quick response code secure: A cryptographically secure anti-phishing tool for QR Codes attacks," in *Computer Network Security: 7th International Conference on Mathematical Methods, Models, and Architectures for Computer Network Security, MMM-ACNS 2017, Warsaw, Poland, August 28-30, 2017, Proceedings 7, 2017*.

[17] M. O. Yadav et al,"Online reservation system using QR Codes based android application system," *International Journal of Scientific and Research Publications*, vol. 4, (12), pp. 1-6, 2014.

[18] (August). "Fitting snake into a QR Codes." Available: https://hackaday.com/2020/08/17/fitting-snake-into-a-qr-code/.

[19] MattKC, "Can you fit a whole game into a QR Codes?," vol. Web, July 31, 2020.

[20] "Open redirect due to scanning QR Codes via brave browser." Available: https://hackerone.com/reports/1946534.

[21] (February,) "What is universal cross-site scripting (UXSS) vulnerability?."Available: https://zofixer.com/what-is-universal-cross-site-scripting-uxss-vulnerability/.

[22] AnonHafiizh."(CVE-2021-43530) - UXSS on QRcode Code Reader (Mozilla android version: 93.2.0 (build #2015839747))," 2021.

[23] A. Pawar et al,"Secure QR Codes scanner to detect malicious URL using machine learning," in *2022 2nd Asian Conference on Innovation in Technology (ASIANCON), 2022*.

[24] F. Vanhoenshoven et al, "Detecting malicious URLs using machine learning techniques," in *2016 IEEE Symposium Series on Computational Intelligence (SSCI), 2016*.

[25] P. Peng et al, "Opening the blackbox of virustotal: Analyzing online phishing scan engines," in *Proceedings of the Internet Measurement Conference, 2019*.

[26] J. O. Okesola et al, "Malvertisements detection using urlscan. io, pulsedive, and SucuriSiteCheck," in *2023 International Conference on Science, Engineering, and Business for Sustainable Development Goals (SEB-SDG), 2023.*

[27] (November 23,). URLScan GmbH. Available: https://urlscan.io/about.

[28] Anonymous "Streamlit: Knowledge Base," unpublished.

[29] C. Bentéjac, A. Csörgő and G. Martínez-Muñoz, "A comparative analysis of gradient boosting algorithms," *Artif. Intell. Rev.*, vol. 54, pp. 1937-1967, 2021.

[30] R. T. Kneusel, *How AI Works: From Sorcery to Science. (1st ed.)* San Francisco: No Starch Press, 2023.

[31] R. Y. Choi et al.,"Introduction to machine learning, neural networks, and deep learning," *Translational Vision Science & Technology*, vol. 9, (2), pp. 14, 2020.

[32] A. Roland, "Open redirect due to scanning QR Codes via brave browser," unpublished.

[33] A. S. Rafsanjani et al., "QsecR: Secure QR Codes Scanner According to a Novel Malicious URL Detection Framework," IEEE Access, 2023.

[34] S. Marchal, J. Francois, R. State, and T. Engel. "PhishStorm: Detecting Phishing with Streaming Analytics." IEEE Transactions on Network and Service Management (TNSM), 11(4):458-471, 2014.

[35] Mohammad Saiful Islam Mamun, Mohammad Ahmad Rathore, Arash Habibi Lashkari, Natalia Stakhanova and Ali A. Ghorbani, "Detecting Malicious URLs Using Lexical Analysis", Network and System Security, Springer International Publishing, pp. 467-482, 2016.

[36] Samaneh Mahdavifar, Nasim Maleki, Arash Habibi Lashkari, Matt Broda, Amir H. Razavi,"Classifying Malicious Domains using DNS Traffic Analysi", The 19th IEEE International Conference on Dependable, Autonomic, and Secure Computing (DASC), Oct. 25-28, 2021, Calgary, Canada.

[37] "Malicious URLs dataset" — Kaggle. Accessed: Nov. 5, 2023. [Online]. Available: https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset/data

[38] "Phishing Site URLs" — Kaggle. Accessed: Nov. 5, 2023. [Online]. Available:https://www.kaggle.com/taruntiwarihp/phishing-site-urls

[39] S. Kumar. Malicious and Benign URLs — Kaggle. Accessed: Nov. 5, 2023. [Online]. Available: https://www.kaggle.com/siddharthkumar25/malicious-and-benign-urls

[40] "Create your free QR Codes"— QR Codes Generator, Accessed: Dec. 3, 2023 [Online]. Available: https://www.qr-code-generator.com/

[41] hc0503, "Laravel-qrcode-campaign: QRcode generate from URL and logo, colors in campaign: Dynamic QRcode Scan: Qrcode hitmap" — GitHub, Accessed: Dec. 3, 2023 [Online] Available: https://github.com/hc0503/laravel7-qrcode-campaign.

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:26:03 UTC from IEEE Xplore. Restrictions apply.